

Blind Image Quality Assessment

CNN vs Vision Transformer

Predire la **qualità percepita** di una foto senza immagine di riferimento

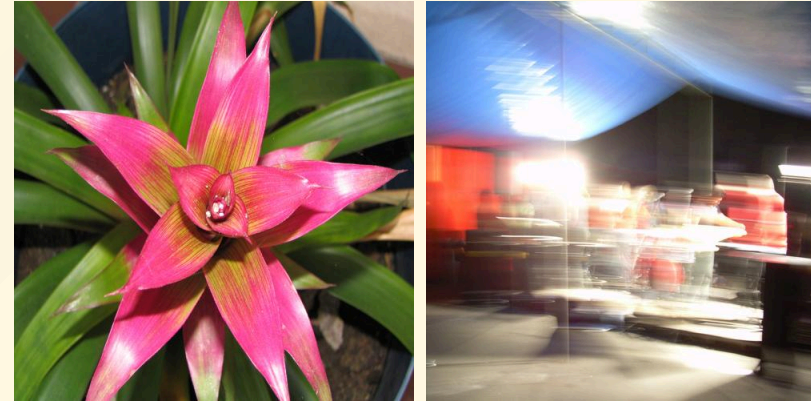
Foundations of Deep Learning

Il task: No-Reference IQA

- **Full-Reference IQA:** confronto con l'originale (PSNR, SSIM) → facile ma irrealistico
- **No-Reference IQA:** solo l'immagine distorta → il modello deve *imparare* cos'è qualità
- Output: **MOS** (Mean Opinion Score), media dei giudizi umani su scala 0–100
- Applicazioni: fotocamere smartphone, compressione adattiva, filtri di upload

Dataset: LIVE In the Wild (ChallengeDB)

- **1169 foto** da dispositivi mobili reali
- Distorsioni **autentiche** e miste (blur, rumore, sovraesposizione, compressione)
- Ogni immagine: **MOS** $\in [0,100]$ + **StdDev** dei giudizi
- Split **stratificato per qualità**, seedato:
818 train / 175 val / 176 test
- Sfida: **dataset piccolo** $\rightarrow$ transfer learning obbligatorio



MOS 92 (ottima) – MOS 6 (pessima)

Metrica target: SRCC

- **SRCC (Spearman)**: correlazione tra i *ranking* → il modello ordina le immagini come gli umani?
- **PLCC (Pearson)**: correlazione lineare dei valori – **RMSE/MAE**: errore assoluto
- Target del progetto: **SRCC** – per le applicazioni conta l'**ordinamento**, non il valore esatto
- Conseguenza: early stopping e model selection su `val_srcc`, **non** su `val_loss`, Keras non ha SRCC built-in → callback custom.

Pipeline dati

- File `.mat` → DataFrame → `tf.data.Dataset`
- MOS normalizzato in **[0,1]** → output con **sigmoide** (predizioni sempre in scala)
- Risoluzione di lavoro: **224 baseline** → **384/500 studiate**
- Augmentation **conservativa**: flip orizzontale, luminosità ± 10 , contrasto 0.9–1.1
- **✗ Vietate**: rotazioni, blur, ricompressione JPEG → *alterano la qualità* → *invalidano il label!*

Crop nativi 384 – preservare il dettaglio (stile DeepBIQ)

- Le distorsioni (rumore, grana, compressione) vivono nei **pixel ad alta frequenza**
- Resize dell'**intera** immagine 500→384 le **sfuma**
- Invece: ritaglio **384×384 dai pixel originali**, *zero resize* → dettaglio 1:1
- 4 crop/immagine = augmentation **spaziale** (MOS ereditato), + solo **flip**

Originale 500×500

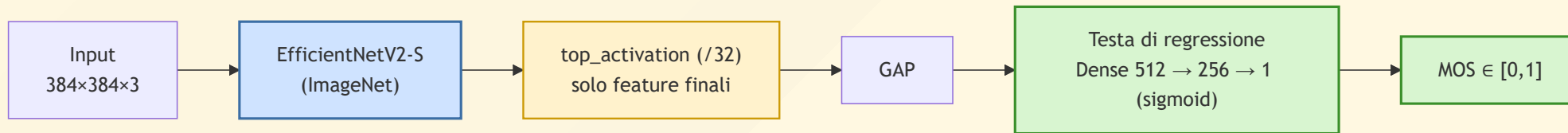
└ resize INTERA → 384	✗ sfuma
└ crop 384×384 nativo	✓ 1:1
(no resampling)	

Test SRCC 0.842 → 0.870

(crop frazione+resize → crop esatto)

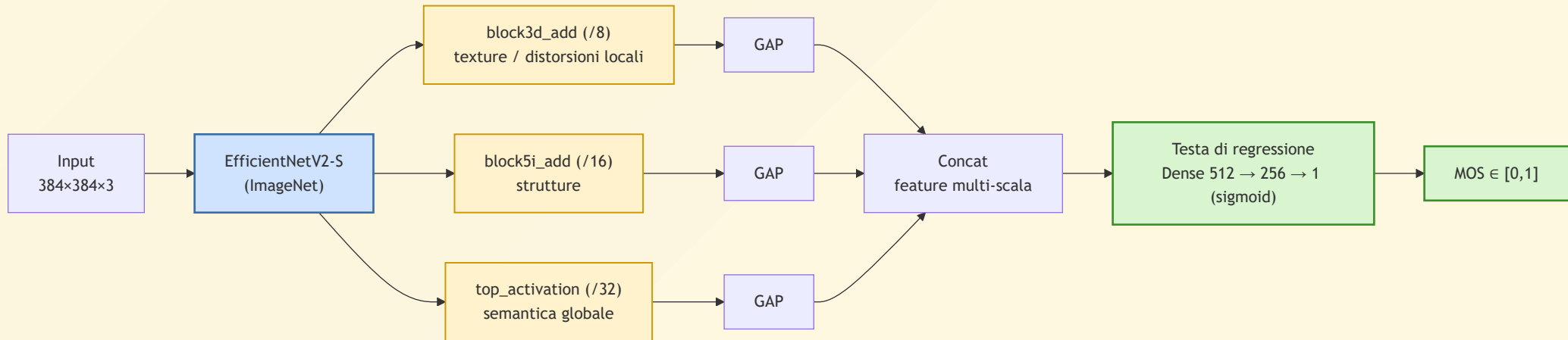
CNN baseline – single-scale

- Backbone **EfficientNet** pre-addestrato ImageNet (provati **B0** ~4M e **V2-S**, più capacità → usato per i risultati di testa)
- **Single-scale**: si usano solo le **feature finali** del backbone → GAP → testa
- Testa di regressione: Dense(512) → Dense(256) → Dense(1, sigmoid)
- È il **riferimento**: backbone forte ma **una sola scala**



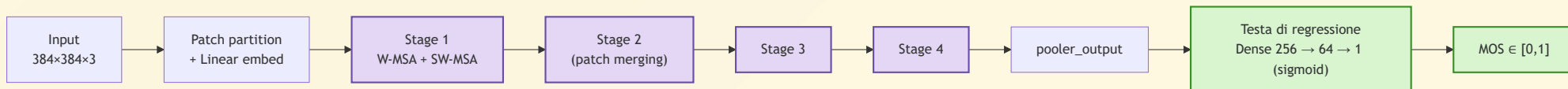
CNN multiscala – la variante "nostra"

- Feature estratte da **3 profondità** del backbone in un solo forward pass
- texture locali (/8) · strutture (/16) · semantica (/32) → **concatenate** prima della testa
- *Ipotesi*: le distorsioni vivono a **scale diverse** (rumore = locale, composizione = globale)



Vision Transformer – Swin

- **Swin Transformer** (HuggingFace, ImageNet): **Swin-Tiny** (~28M) e **Swin-Base @384** (~88M, testa)
- Attention su **finestre locali shiftate** + gerarchia a **4 stadi** → costo lineare, piramide stile CNN
- Testa: Dense(256, GELU) → Dense(64, GELU) → Dense(1) su `pooler_output`
- **Domanda di ricerca:** l'*inductive bias* convolutivo è un vantaggio su dataset piccolo?



Training: fine-tuning progressivo

Fase	Cosa si allena	LR
Phase 1	solo la testa (backbone congelato)	1e-3
Phase 2a	layer alti (top-30 CNN / stadi 2-3 Swin)	basso
Phase 2b	backbone (quasi) intero	ridotto

- Early stopping su `val_srcc` + ReduceLROnPlateau + checkpoint del best
- Ottimizzatori: **Adam** (CNN) / **AdamW + gradient clipping** (Swin, $2e-5 \rightarrow 1e-5$)
- Swin-Base @384: **graph mode** + Phase 2b limitato agli stadi 2-3 (~96% param) per non andare OOM

Funzioni di loss

- **Base:** MSE sul MOS normalizzato
- **MSE + $\lambda \cdot (1 - \text{PLCC di batch})$:** termine di correlazione differenziabile → ottimizza ciò che misuriamo ($\lambda=0.5$ per Swin)
- **Weighted MSE:** peso $w = \exp(-\alpha \cdot \text{std})$ → errori su giudizi *concordi* pesano di più; label rumorose pesano meno



Caso di studio: i bug del `trainable`

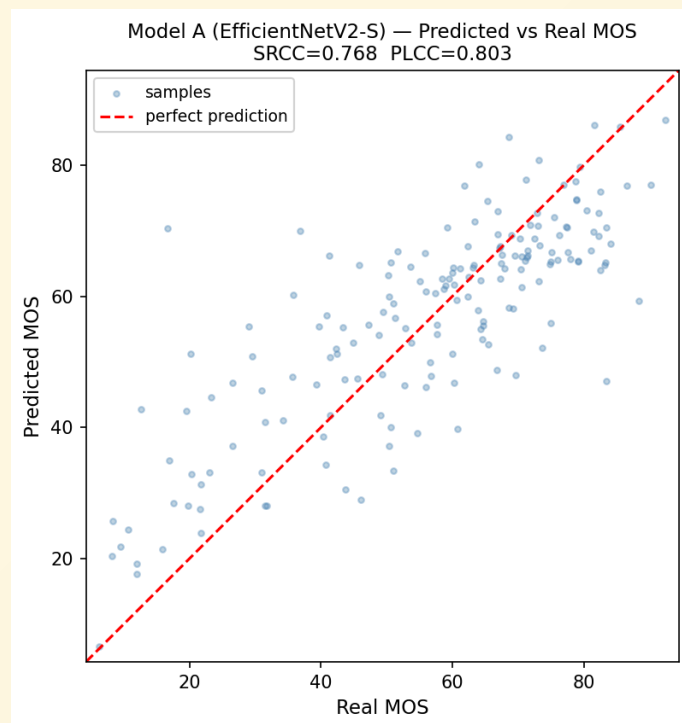
- **Sintomo:** lo Swin sembrava bloccato → "i transformer servono più dati"?
- **Diagnosi reale:** bug *silenzioso* del framework, non i dati
 - **tf-keras (Keras 2):** padre con `trainable=False` **nasconde i figli** → backbone Swin mai fine-tunato
 - **Keras 3 (CNN):** rimettere `backbone.trainable=True` **non** resetta i flag dei figli → si allenava solo il top-30
- **Aggravante:** regola LR ereditata $(lr/10) = 5e-7 \approx \text{zero}$ per lo Swin → passaggio ad AdamW
- **Lezione:** *loggarne sempre i parametri allenabili per fase* + script di verifica

Risultati principali (tutti @384)

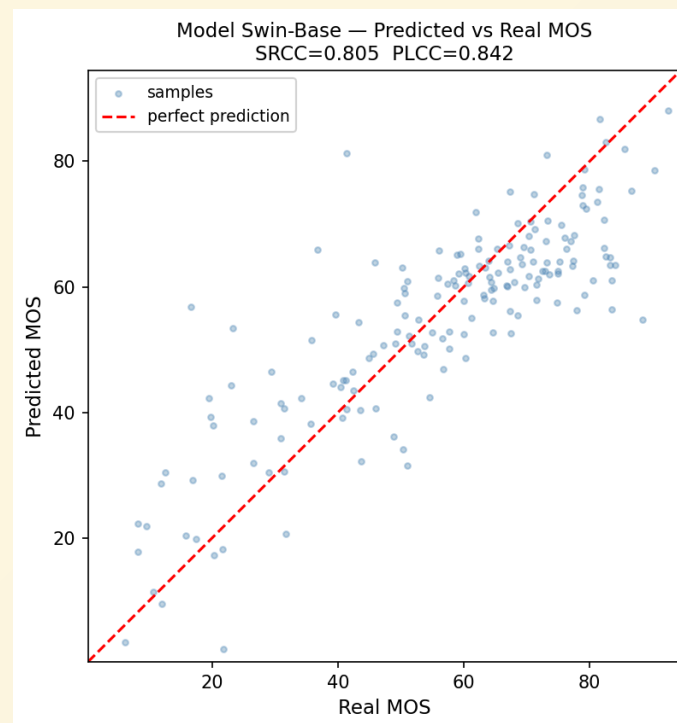
Modello	Backbone	Test SRCC	Test PLCC	RMSE
Single-scale + crop nativi	EfficientNetV2-S	0.768	0.803	12.41
ViT + crop nativi	Swin-Base	0.805	0.842	11.35
Multiscala + crop nativi	EfficientNetV2-S	0.870	0.887	10.47

- **Esperimento a variabile singola:** le due CNN sono identiche (V2S, 384, crop nativi 384×384, stessa ricetta) → cambia **solo** l'estrazione multiscala
- **+0.102 SRCC** (0.768 → 0.870) attribuibili al solo multiscala – miglior modello singolo del progetto
- Il ViT (Swin-Base, stesso 384 + crop nativi) è competitivo ma sotto: **0.805** – anche qui i crop nativi aiutano (+0.041 sul Swin senza crop)

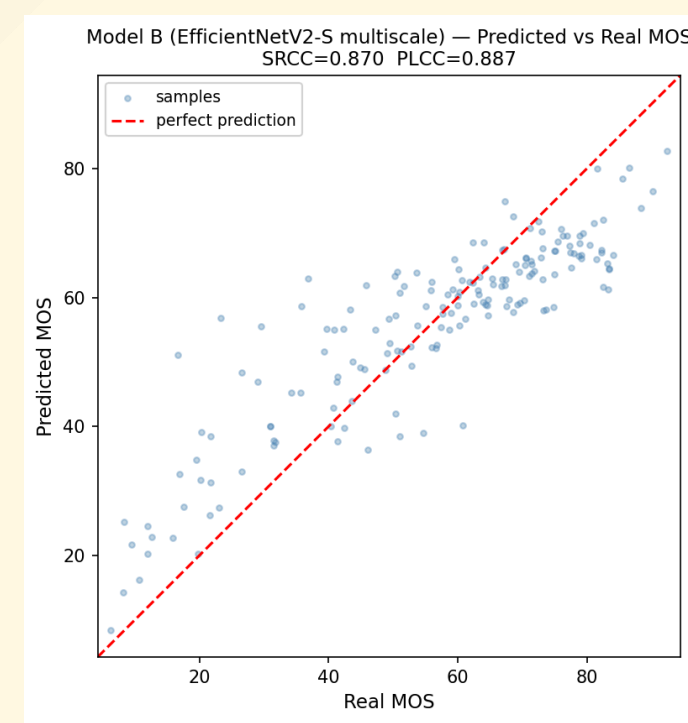
Risultati: predetto vs reale (test set)



Single-scale V2S + crop nativi — 0.768



ViT Swin-Base + crop nativi — 0.805



Multiscala V2S + crop nativi — 0.870

Ablation: le due leve

Leva 1 – Multiscala (a parità di tutto il resto)

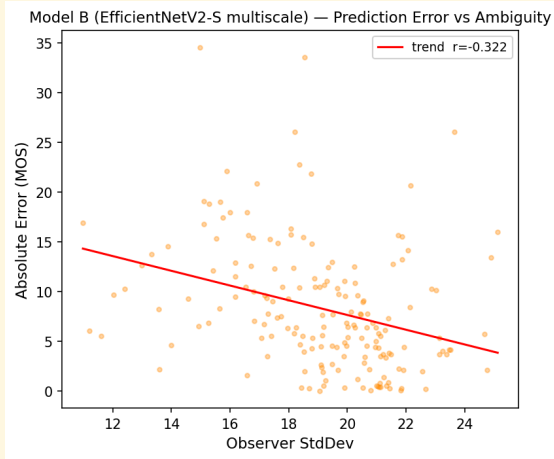
SRCC	
V2S single @384 + crop nativi	0.768
V2S multi @384 + crop nativi	0.870 (+0.102)
B0 single @224	0.630
B0 multi @224	0.770 (+0.140)

Leva 2 – Risoluzione (baseline 224)

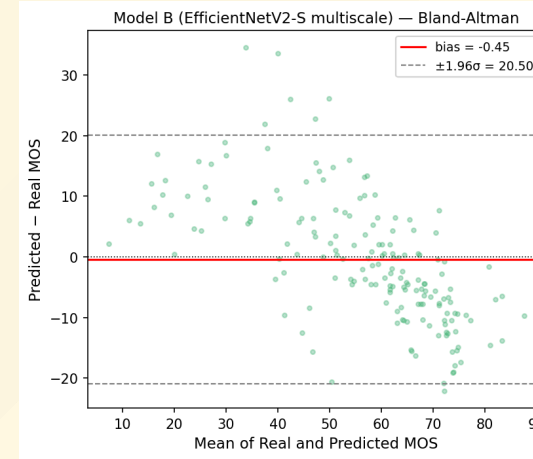
Modello	224	384	500
Multi B0	.770	.778	.789
V2S single	.620	.729	–

Miglioramento **monotono**; limite = memoria GPU

Analisi degli errori (multiscala V2S)



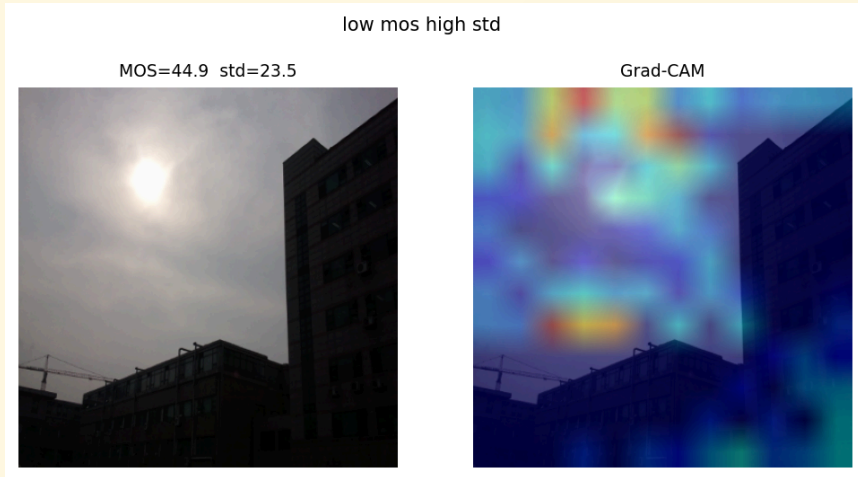
Errore vs StdDev osservatori



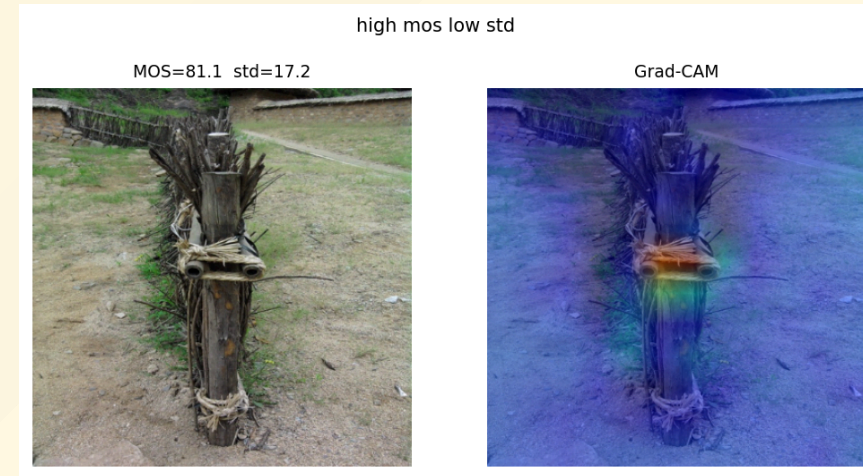
Bland-Altman

- Contrariamente all'attesa, l'**errore assoluto DIMINUISCE** col disaccordo tra osservatori (**$r = -0.322$**)
- Le immagini a basso StdDev sono spesso proprio gli **estremi** di qualità (consenso forte) → lì l'errore assoluto è più alto: l'errore è guidato dalla **saturazione**, non dall'ambiguità
- Cala invece la **qualità di ranking** sulle immagini ad alto disaccordo: SRCC **0.88** → **0.72** (low → high std)

Interpretabilità: Grad-CAM (multiscala V2S)



MOS basso – il modello si concentra sulle regioni degradate



MOS alto – nessun difetto da segnalare, l'attenzione segue il soggetto

- *Dove guarda* la CNN: su immagini degradate le attivazioni si concentrano sulle zone **sfocate / rumorose / bruciate**; su immagini di qualità (niente difetti) l'attenzione segue il **soggetto** → comportamento sensato, niente scorciatoia statistica
- Limite: non applicabile a Swin → future work (occlusion / attention maps)

Conclusioni

- **Miglior modello singolo:** V2S multiscala @384 + **crop nativi** → **SRCC 0.870**
- Le leve che contano: **multiscala** (+0.102) · **risoluzione** · **crop nativi** (384×384 dai pixel originali, zero resampling)
- ViT Swin-Base @384 + **crop nativi**: **0.805** — competitivo ma sotto al multiscala
- **L'inductive bias multiscala resta un vantaggio** a questa scala di dati

Future work: pre-training su KonIQ-10k · ensemble CNN+Swin · multi-seed CI · interpretabilità Swin